

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	$s = x \oplus y$ $c = x \& y$	Dodaje dwa bity. <code>s</code> to suma, <code>c</code> to bit przeniesienia (carry).
Pełny sumator (FA)	$s = x \oplus y \oplus c_i$ $c_o = x \& y \mid c_i \& (x \oplus y)$	Jak półsumator, ale przyjmuje też przeniesienie wejściowe <code>c_i</code> .
Sumator n-bitowy	$n \times \text{FA}$	Kaskadowo wyjście C_i trafia na wejście kolejnego FA. Ostatnie <code>c_o</code> to Carry Out całości.

Układy kombinacyjne		
Dekoder	$n \rightarrow 2^n$	Wejście koduje liczbę k . Na wyjściu zapalony jest bit k .
Multiplekser	$2^n + n \rightarrow 1$	2^n bitów danych + n bitów sterujących S . Na wyjściu zapalony S -ty bit danych.
ALU	$A, B, f \rightarrow C$	Dekoder na bitach f wybiera operację np. $A + B$, multipleksowanie wyników bramkami AND/OR.

Układy sekwencyjne (pamięć)		
Przerzutnik S-R	<code>S</code> - set, <code>R</code> - reset	Przechowuje 1 bit (Q). <code>S=1</code> ustawia $Q = 1$, <code>R=1</code> zeruje, <code>S=R=0</code> trzyma stan. Dwa sprzężone <code>NOR</code> -y.
Sterowany poziomem	aktywny gdy <code>CLK</code> = 1	Wejścia <code>S</code> / <code>R</code> zAND-owane z zegarem.
Sterowany zboczem	zbocze 1 $\rightarrow$ 0	Dwa przerzutniki poziomowe w kaskadzie. Stan przepisuje się w momencie zmiany zegara.

2. Typy danych

Typ	<code>char</code>	<code>short</code>	<code>unsigned</code>	<code>int</code>	<code>long</code>	<code>float</code>	<code>double</code>	<code>void*</code>
x86-64	1	2	4	4	8	4	8	8

3. Rozszerzanie, obcinanie i przesunięcia

<code>x << k</code>	$x \cdot 2^k$ (sgn./uns.)
<code>u >> k</code>	$\lfloor u/2^k \rfloor$ - logiczne (zera)
<code>x >> k</code>	arytmetyczne (kopia MSB), zaokr. do $-\infty$
$x/2^k$ do 0	<code>(x + (1<<k)-1) >> k</code>
Strength reduction	<code>x*24</code> $\rightarrow$ <code>(x<<5)-(x<<3)</code>

4. Operacje i endianness

Negacja U2

-x = ~x+1 ; -TMin=TMin , -0=0

Wykr. nadmiaru (s=x+y)

((s^x)&(s^y))>>(w-1)

Maska / abs

m=x>>31; abs=(x^m)-m

c ? a : b

b ^ (-c & (a ^ b))

Promocja w C:

unsigned i int w jednym wyrażeniu/porównaniu

(< > ==) → int promowany do unsigned (bity bez zmian, -1 → UMax).

Schemat

Najniższy adres

0x0123

Big Endian

MSB

[01][23]

Little Endian

LSB

[23][01]